Manuel d'installation et de configuration - pili-pos v1.0.0

Ce manuel s'adresse au technicien installateur (= distributeur, nuagemagique.dev). Il décrit la mise en place complète d'un kiosque pili-pos chez un client HoReCa.

1. Matériel requis

Élément	Modèle / spécifications	Qté
Terminal POS	Penetek A3216 M82 ou Advantech USC-360J (Intel J6412, 8 Go RAM, 128 Go SSD, écran 15,6" 1920×1080 PCAP, IP65)	1
Boîtier FDM	FDM module2 (PRACSYS, certifié SCE 2.0) ou autre FDM certifié SCE 2.0	1
Imprimante thermique	Epson TM-m30III, raccordée au LAN (le POS imprime en TCP brut, port 9100 ; l'USB n'est pas pris en charge)	1
Tiroir-caisse	Standard, connexion RJ11 vers l'imprimante	1
Câble Ethernet	RJ45 Cat-5e ou supérieur	3 (POS, FDM, imprimante)
Switch / box internet	4 ports minimum	1
Clé USB d'installation/mise à jour	8 Go minimum - l'image kiosque complète (`pili-pos-kiosk-<profil>.img.xz`) y est écrite telle quelle (§2), pas un simple installeur FAT32	1
Onduleur (recommandé)	600 VA minimum	1

Puce TPM 2.0 obligatoire (fTPM Intel PTT intégré au J6412, ou puce dTPM discrète selon la carte mère) : le premier démarrage chiffre `/var` (base fiscale) en LUKS2 avec déverrouillage automatique lié à ce TPM (§3.4, `SECURITE_BD_FR.md` §3.6). Sans TPM2 activé en BIOS, le premier démarrage échoue à l'étape d'enrôlement TPM2 - vérifier et activer avant de commencer (§3.1).

2. Préparation de la clé USB d'installation

2.1 Télécharger l'image kiosque

L'image est fournie par le distributeur sous forme d'un fichier `pili-pos-kiosk-vX.Y.Z.img.xz`. Sur la machine du distributeur :

```
xz -d pili-pos-kiosk-vX.Y.Z.img.xz
```

2.2 Écrire l'image sur la clé USB

Attention : **L'opération efface toute la clé USB.**

Sur Linux/macOS :

```
# Identifier la clé (souvent /dev/sdX ou /dev/diskN)
lsblk        # Linux
diskutil list # macOS
```

```
# Écrire l'image (remplacer /dev/sdX par le bon device)
sudo dd if=pili-pos-kiosk-vX.Y.Z.img of=/dev/sdX bs=4M status=progress conv=fsync
```

Sur Windows : utiliser **Rufus** ou **Balena Etcher** avec l'image.

3. Installation sur le terminal POS

3.1 Configuration BIOS

À la première mise sous tension du terminal :

1. Entrer dans le BIOS (touche `Del` ou `F2` selon le modèle).
2. Activer **Boot from USB**.
3. **Activer le TPM 2.0** (souvent affiché "Intel PTT" ou "fTPM" sur le J6412) - obligatoire pour le chiffrement automatique de `/var` au premier démarrage (§3.4, §1). Sans cela, `pili-provision` échoue à l'enrôlement TPM2 et le terminal reste bloqué avant d'atteindre l'écran de connexion.
4. **Laisser Secure Boot désactivé** (le paramètre par défaut sur la plupart de ces cartes mères) : l'image ne signe pas encore son UKI (Unified Kernel Image) contre une clé Secure Boot - l'activer empêcherait le terminal de démarrer. Voir `SECURITE_BD_FR.md` §3.6 pour ce que cela implique pour le scellement TPM2.
5. **Définir un mot de passe BIOS** (à conserver précieusement - le client n'en a pas besoin) et verrouiller l'ordre de boot sur le disque interne pour l'usage normal (USB seulement au besoin, via le mot de passe BIOS).
6. Sauvegarder et redémarrer.

3.2 Boot sur la clé USB

1. Brancher la clé USB préparée (image kiosque écrite telle quelle, §2 - elle contient déjà les deux entrées de menu suivantes, ajoutées par `deploy/image/build.sh` en dupliquant l'UKI construit par mkosi avec une ligne de commande différente).
2. Démarrer le terminal - il doit booter sur la clé (menu `systemd-boot`).
3. Choisir l'entrée **"Install"** pour une installation neuve, ou **"Update"** pour mettre à jour un terminal déjà en service sans perdre `/var` (base fiscale + sauvegardes) - voir §6.

3.3 Procédure d'installation

Le service `pili-installer.service`, actif uniquement quand le noyau a démarré avec `pili.installer=install` (l'entrée "Install" du menu), fait le travail :

1. Détection du disque interne (le premier disque autre que la clé USB elle-même).
2. **Install** : réplication complète de la table de partitions de la clé (`sgdisk --replicate` + `-G` pour de nouveaux UUID) puis copie partition par partition (ESP + racine ; la partition `pili-data` copiée est vide/non formatée à ce stade) vers le disque interne. **Update** : seules les partitions ESP et racine sont recopiées (`dd` partition à partition) ; la partition `pili-data` déjà présente sur le disque interne (avec son contenu chiffré) n'est **jamais touchée** - c'est ce qui préserve `pos.db` d'une mise à jour à l'autre.
3. Le disque interne contient maintenant Debian 13 minimal + pili-pos
   - Cage + RustDesk (extrait) + les outils de provisioning, en lecture seule pour la racine - mais rien n'est encore déchiffré ni configuré : cette étape ne fait que poser les partitions ESP/racine sur le disque interne.

Durée approximative : 3 à 6 minutes (copie de partitions brutes, beaucoup plus rapide que l'ancienne installation "à chaud" sur un Debian nu).

3.4 Premier démarrage - chiffrement, mots de passe, clé de récupération

Retirer la clé USB et redémarrer. Le terminal doit :

1. Démarrer sur le disque interne (systemd-boot $\rightarrow$ UKI).

2. `systemd-repart` finalise la partition `pili-data` (taille = tout l'espace libre restant sur le disque).
3. `pili-provision.service` s'exécute automatiquement, **une seule fois** (il ne se relance jamais aux démarrages suivants) :
   - Formate `pili-data` en LUKS2 avec une clé temporaire.
   - Enrôle le TPM2 (déverrouillage automatique, silencieux, à chaque démarrage normal) et une **clé de récupération**.
   - Supprime la clé temporaire.
   - Formate le volume déchiffré en ext4, le monte sur `/var`.
   - Copie la base et le `pos.json` d'usine, y inscrit le `device_id` (adresse MAC de la carte réseau du terminal, en minuscules - voir §4.2 ; jamais `DEV001` ni une valeur libre).
   - Génère un mot de passe `root` aléatoire (et, sur l'image d'admission, un mot de passe `spf`), les affiche **une seule fois** à l'écran avec la clé de récupération LUKS, puis attend une pression sur Entrée.
4. **Inscrire immédiatement** le mot de passe `root` (et `spf` le cas échéant) ainsi que la clé de récupération LUKS dans le carnet du distributeur (jamais communiqués au client) - ces valeurs ne sont **plus jamais réaffichées** et ne sont conservées nulle part sur le disque après cet écran.
5. Une fois Entrée pressée, le terminal poursuit son démarrage : Cage Wayland en plein écran, puis pili-pos en mode kiosque.
6. Écran de connexion affiché (avec un message rouge "POS NON CONFIGURÉ" tant que les champs propres à l'établissement de `pos.json` - §4.2 - n'ont pas été renseignés).

Durée approximative de cette étape : 1 à 3 minutes (l'enrôlement TPM2 est l'opération la plus lente).

Reprise après coupure de courant pendant cette étape : si le terminal est mis hors tension entre l'enrôlement TPM2 et le montage de `/var`, `pili-provision` détecte au redémarrage suivant un volume LUKS2 déjà enrôlé TPM2 mais non monté, et se contente de le déverrouiller/monter (aucune re-génération de mots de passe - ils n'auraient de toute façon plus la même valeur que ceux déjà notés). Si le volume est LUKS2 mais **sans** jeton TPM2 (coupure encore plus tôt), le script s'arrête et affiche qu'une intervention manuelle est nécessaire depuis la console de maintenance (§4.1) - ce cas n'a pas été exercé sur matériel réel au moment de la rédaction de ce document (voir le rapport final Phase 4).

4. Configuration initiale

4.1 Accès maintenance

Le terminal n'a **pas de TTY accessible** par défaut. Pour la maintenance initiale, le distributeur :

1. Branche un clavier USB sur le terminal.
2. Presse `Ctrl + Alt + F2` (séquence cachée non accessible à l'opérateur).
3. Se connecte en `root` avec le mot de passe noté à l'installation.

4.2 Fichier de configuration `/etc/pili-pos/pos.json`

Ce fichier **existe déjà** à ce stade : `pili-provision.service` l'a copié depuis le gabarit d'usine au premier démarrage (§3.4) et y a déjà inscrit le `device_id` réel du terminal. `/etc/pili-pos` est un lien symbolique vers `/var/lib/pili-etc/pili-pos` (partition chiffrée LUKS2, §3.4) - il ne redevient lisible qu'après déverrouillage, donc uniquement une fois le terminal démarré. Reste à compléter les champs propres à l'établissement.

Éditer avec `vi /etc/pili-pos/pos.json` (l'éditeur `vim-tiny` est installé sur l'image précisément pour cette étape) :

```
{
  "pos_id": "CFOD0061234567",
  "terminal_id": "POS1",
  "device_id": "00:1b:44:11:3a:b7",
  "restaurant_name": "Restaurant Exemple",
  "restaurant_legal_name": "Exemple Horeca SRL",
  "restaurant_address": "Rue de l'Exemple 1, 4000 Liège",
  "restaurant_vat": "BE0499999960",
```

```
  "restaurant_est_no": "2123456791",
  "fdm_url": "http://192.168.1.20:8080/graphql",
  "printer_address": "192.168.1.30:9100",
  "kitchen_printer_address": "192.168.1.31:9100",
  "training_mode": false,
  "day_end_hour": 5,
  "language": "fr",
  "backup_dir": "/var/lib/pili-pos/backups"
}
```

Champs :

Champ	Description	Source
`pos_id`	Identifiant POS : 14 caractères, lettres majuscules ou chiffres (commence par `C`). Attribué par le SPF Finances à la certification.	SPF Finances
`terminal_id`	ID du terminal physique. Libre, en général `POS1`, `POS2`…	Distributeur
`device_id`	Adresse MAC de la carte réseau du terminal (minuscules, séparée par `:`), imprimée sur chaque ticket (manuel §VI, ligne “Appareil: ”). Renseignée **automatiquement** par `pili-provision` au premier démarrage (interface de la route par défaut, sinon la première interface Ethernet non-`lo` - voir `deploy/provision/provision.sh::detect_device_id`) ; ne pas modifier à la main sauf changement matériel de la carte réseau.	Automatique (terminal)
`restaurant_name`	Nom commercial du restaurant tel qu’apparaît sur le ticket	Client
`restaurant_legal_name`	Facultatif. Dénomination légale de la société exploitante, imprimée en bas du ticket (« Dénomination légale : »), avec l’adresse et le numéro de TVA. Absent : le nom commercial est imprimé.	Client
`restaurant_address`	Adresse complète	Client
`restaurant_vat`	Numéro de TVA belge : `BE` suivi de 10 chiffres commençant par 0 ou 1, valide modulo 97 (ex. `BE0499999960`)	Client
`restaurant_est_no`	Numéro d’unité d’établissement : 10 chiffres commençant par 2 à 8, valide modulo 97 (ex. 2123456791)	BCE / client
`fdm_url`	URL GraphQL du FDM sur le LAN. FDM module2 : `http://<IP du FDM>/graphql/` (port 80)	Configuration FDM

Champ	Description	Source
`fdm_token_file`	Facultatif. Fichier contenant le seul jeton d'accès (Bearer) d'un FDM qui en exige un (certains modèles, y compris sur le LAN : le jeton se lit sur la page de paramètres du FDM) ; jamais le jeton lui-même dans `pos.json`. Absent si le FDM n'en demande pas.	Configuration FDM
`fdm_ca_file`	Requis si `fdm_url` est en `https://`. Autorité qui signe le certificat du FDM (PEM ; `ca.crt` de module2).	Configuration FDM
`fdm_client_cert_file`, `fdm_client_key_file`	Ensemble ou pas du tout. Certificat client et sa clé (PEM ; `client.crt` et `client.key` de module2) pour un FDM en mTLS. Un fichier illisible s'affiche dans le bandeau des problèmes de configuration.	Configuration FDM
`printer_address`	IP:port de l'imprimante sur le LAN (port 9100 par défaut)	Configuration imprimante
`kitchen_printer_address`	Facultatif. IP:port de l'imprimante du bar ou de la cuisine, qui reçoit le bon de commande (sans prix, « ceci n'est pas un ticket de caisse TVA valable ») des lignes envoyées. Absent : le bon sort sur `printer_address`.	Configuration imprimante
`training_mode`	`false` en production. À NE JAMAIS laisser `true` après installation.	Distributeur
`hubrise_enabled`	Facultatif, `false` par défaut. `true` : les commandes en ligne (plateformes de livraison, webshop, par HubRise) sont activées dès le démarrage. Un Gérant peut ensuite les activer ou les couper dans Réglages ; chaque changement est journalisé et le dernier l'emporte. Voir `COMMANDES_EN_LIGNE_FR.md`.	Client
`day_end_hour`	Heure (0 à 12, heure belge) à laquelle la journée commerciale se termine ; défaut `5`. Un service qui dépasse minuit reste sur sa journée jusqu'à cette heure. Une journée restée ouverte au-delà est clôturée par un rapport Z automatique (FAQ producteurs Q501).	Client

Champ	Description	Source
`language`	Langue par défaut de l'interface : `fr` / `nl` / `de` / `en` / `zh`. Les mentions fiscales imprimées suivent cette clé, restreinte aux langues nationales (fr/nl/de) - toute autre valeur émet le ticket en FR.	Client
`backup_dir`	Répertoire des sauvegardes (défaut : `/var/lib/pili-pos/backups`)	Distributeur

Vérification Le FDM refuse tout événement dont l'identification est mal formée (description détaillée POS v2.2 §2.2.2). Après l'édition, vérifier :

```
/opt/pili-pos/pili-pos --check-config
```

La commande lit `pos.json` comme le POS et affiche `configuration OK`, ou la liste des champs à corriger (code de sortie 1) : format du `pos_id`, numéros de TVA et d'établissement (longueur, premier chiffre, modulo 97), champs vides, URL du FDM, langue, bloc `digital_ticket`. Tant qu'un champ est refusé, le POS le signale aussi dans un bandeau rouge au bas de l'écran et dans le journal (`journalctl -u pili-pos`).

Langue du ticket La clé `language` détermine la langue des mentions fiscales imprimées. Elle est fixée à l'installation d'après la **région linguistique de l'établissement** où le POS est exploité (manuel de certification §VI, p. 99) - ni d'après le siège du distributeur, ni d'après la langue de l'interface souhaitée par le personnel :

Région linguistique de l'établissement	`language`
Région de langue française	`fr`
Région de langue néerlandaise	`nl`
Région de langue allemande	`de`
Région bilingue de Bruxelles-Capitale	`fr` ou `nl`

Exemple : un établissement à 4000 Liège (région de langue française) → `"language": "fr"`. Voir `GARANTIE_MAJ_LANGUES_FR.md` §3.2.

Ticket numérique par e-mail (optionnel) Sans bloc `digital_ticket`, le POS n'émet que des tickets papier et n'affiche pas le choix du support. Pour proposer le ticket par e-mail (manuel de certification §III.H), ajouter à `pos.json` :

```
"digital_ticket": {
  "project_id": "<identifiant du projet Scaleway>",
  "from_email": "tickets@<domaine vérifié>",
  "from_name": "<nom du restaurant>",
  "api_key_file": "/etc/pili-pos/secrets/tem.key",
  "allow_digital_only": false
}
```

Champ	Description
`project_id`	Projet Scaleway du service Transactional Email.
`from_email`	Adresse d'expédition, sur un domaine vérifié dans Scaleway.
`from_name`	Nom affiché comme expéditeur (en général le nom du restaurant).

Champ	Description
`api_key_file`	Fichier contenant la seule clé secrète de l'API, jamais la clé elle-même dans `pos.json`.
`allow_digital_only`	`true` : l'opérateur peut choisir « E-mail » seul, sans papier. `false` (défaut) : seuls « Papier » et « Les deux » sont proposés.
`region`	Facultatif, `fr-par` par défaut.

Installer la clé, lisible par le seul service POS :

```
install -d -m 750 -o root -g pos /etc/pili-pos/secrets
(umask 027; cat > /etc/pili-pos/secrets/tem.key)   # coller la clé, Entrée, puis Ctrl+D
chown root:pos /etc/pili-pos/secrets/tem.key
systemctl restart pili-pos
```

Le seul trafic sortant ajouté est HTTPS (443) vers `api.scaleway.com`. Si la clé est absente ou illisible, le POS le consigne dans son journal et reste en papier seul. Si le service ne répond pas, le choix « E-mail » seul devient « Les deux » : le client reçoit le ticket papier tout de suite et l'e-mail dès le retour du service (`TICKET_NUMERIQUE_FR.md`).

Commandes en ligne (HubRise). Sans bloc `hubrise`, la caisse ne peut pas se connecter à HubRise (« HubRise n'est pas configuré sur cette caisse »). Pour un établissement qui reçoit des commandes en ligne (`COMMANDES_EN_LIGNE_FR.md`), ajouter à `pos.json` le client OAuth HubRise fourni par le distributeur :

```
"hubrise_enabled": true,
"hubrise": {
  "client_id": "<identifiant>.clients.hubrise.com",
  "client_secret_file": "/etc/pili-pos/secrets/hubrise.secret"
}
```

Champ	Description
`client_id`	Identifiant du client OAuth HubRise de la caisse.
`client_secret_file`	Fichier contenant le seul secret du client, jamais le secret lui-même dans `pos.json`.
`oauth_base_url`	Facultatif, `https://manager.hubrise.com/oauth2/v1` par défaut.
`api_base_url`	Facultatif, `https://api.hubrise.com/v1` par défaut.
`redirect_uri`	Facultatif, `urn:ietf:wg:oauth:2.0:oob` par défaut : HubRise affiche un code à recopier dans la caisse.
`scope`	Facultatif, `location[orders.write,customer_list.read,catalog.read]` par défaut.

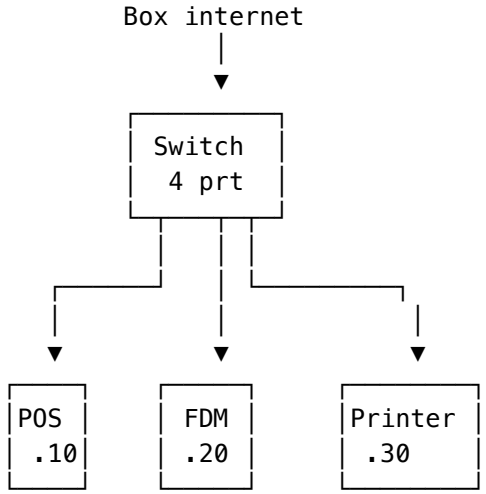
Installer le secret comme la clé du ticket numérique :

```
(umask 027; cat > /etc/pili-pos/secrets/hubrise.secret)   # coller le secret, Entrée, puis Ctrl+D
chown root:pos /etc/pili-pos/secrets/hubrise.secret
pili-pos --check-config        # signale un secret absent ou vide
systemctl restart pili-pos
```

Puis, connecté en Gérant : Réglages, section HubRise, « Se connecter à HubRise ». La caisse n'a pas de navigateur : elle affiche un QR code (et le lien en clair). Le scanner avec un téléphone, se connecter à HubRise, choisir l'établissement, puis taper dans la caisse le code affiché par HubRise et toucher « Valider la connexion ». Le trafic sortant ajouté est HTTPS (443) vers `manager.hubrise.com` et `api.hubrise.com`.

4.3 Configuration réseau LAN

Le POS et le FDM doivent être sur le **même LAN privé**. Schéma type :



Adresses IP statiques recommandées (subnet `192.168.1.0/24`) : - POS : `192.168.1.10` - FDM : `192.168.1.20:8080` (port §4.2 ci-dessus) - Imprimante : `192.168.1.30:9100`

(Réservations DHCP sur le routeur, cf. `SCHEMA_ASSEMBLAGE_FR.md` §4.) - Box : `192.168.1.1`

4.4 Configuration utilisateurs

Créer les utilisateurs initiaux dans la base SQLite :

```
sqlite3 /var/lib/pili-pos/pos.db
```

```
-- Gérant
INSERT INTO users (insz, name, role, active)
VALUES ('XXXXXXXXXXX', 'Bruno Moyse', 'admin', 1);

-- Serveur
INSERT INTO users (insz, name, role, active)
VALUES ('YYYYYYYYYYY', 'Marie Dupont', 'cashier', 1);

-- Apprenti
INSERT INTO users (insz, name, role, active)
VALUES ('ZZZZZZZZZZZ', 'Pierre Tran', 'waiter', 1);
```

Aucun PIN n'est saisi ici. Le Gérant choisit le sien à sa première connexion : sa tuile ouvre "Choisissez votre PIN" (un Gérant ne peut pas se connecter sans PIN). Le Serveur et l'Apprenti fixent le leur en touchant leur nom sur l'écran de commande. Ne pas créer l'INSZ `00000000029` : c'est l'identité système, qui ne se connecte jamais.

(Voir `MANUEL_PROGRAMMATION_FR.md` pour la création/modification de menus, modificateurs etc.)

4.5 Synchronisation horaire

Vérifier que `chrony` est actif :

```
systemctl status chrony
chronyc tracking
```

L'horloge **doit être à la seconde près**. Une dérive crée des ennuis à la signature FDM (qui exige un `posDateTime` cohérent).

4.6 Test du flux complet

Avant de quitter, effectuer un test de bout en bout :

1. Redémarrer le terminal (`reboot`).
2. Se connecter avec un INSZ valide.
3. Vérifier que le pointage S signe correctement.
4. Ouvrir une table, ajouter un item.
5. Encaisser en espèces - le ticket doit s'imprimer avec QR code et le tiroir s'ouvrir.
6. Inspecter `fiscal_events` :

```
SELECT id, event_type, sequence_number, short_signature
FROM fiscal_events ORDER BY id DESC LIMIT 5;
```

Trois lignes minimum : un S (work-in), un N (la vente), idéalement un P si la commande a été envoyée en cuisine.

7. Générer un rapport X pour validation finale.

5. Vérification de l'intégrité du binaire

Le hash SHA-1 du binaire `pili-pos` certifié est consigné dans le formulaire SPF Finances. Vérifier :

```
sha1sum /opt/pili-pos/pili-pos
```

La valeur doit correspondre exactement à celle déclarée dans le formulaire. Si différente : binaire non-certifié, **NE PAS METTRE EN PRODUCTION**.

6. Mise à jour (procédure post-cert)

Une mise à jour suit la procédure officielle SPF Finances (`Manuel_Intro_ToRead_FR.md` §C) :

1. Le distributeur déclare la modification au SPF Finances :
   - Changelog signé
   - Affirmation qu'aucune autre modification n'a été apportée
   - Nouveau hash SHA-1
2. Délai de traitement SPF : 15 jours.
3. Après accord, le distributeur peut déployer sur les sites clients.

Procédure technique de déploiement :

1. Remettre le binaire `vX.Y.Z+1` sur le terminal : clé USB sur site, ou transfert de fichiers pendant une session RustDesk ouverte à la demande par un responsable sur place (`deploy/REMOTE-OPS.md`).
2. Accès maintenance (`Ctrl+Alt+F2`, login `root`), puis :

   ```
   /opt/pili-pos/deploy/update.sh /root/pili-pos --sha1 <SHA-1 déclaré>
   ```

   Le script refuse tout binaire dont le SHA-1 diffère de la valeur déclarée. Les étapes 3 à 7 ci-dessous décrivent la variante « réinstallation complète de l'image » depuis une clé USB.
3. Sauvegarder la base courante :

   ```
   sqlite3 /var/lib/pili-pos/pos.db ".backup /tmp/pos-before-update.db"
   ```
4. Booter sur la clé USB en mode "Update" (sélection menu).
5. L'updater **conserve** le contenu de `/var/lib/pili-pos/` (base + sauvegardes + logs) et remplace uniquement le binaire et les ressources.
6. Redémarrer.
7. Vérifier le nouveau hash SHA-1 et que `fiscal_events` est intacte.

7. Annexes

7.1 Liste des numéros INSZ de test

Les INSZ de test SPF (depuis `UC_TestSetup_20240806.md`) :

Rôle	INSZ
Gérant	00000000196
Serveur	00000000295
Apprenti	00000000394
Robot (automate, identité système, sans connexion)	00000000029

À utiliser exclusivement lors de la phase de test pré-soumission.

7.2 Numéros pos_id de test

`CP0S0031234567` - le posId de la configuration de test SPF Finances (UC_TestSetup_20240806), autorisé sur le simulateur module2 de la machine de test.

7.3 FDM

- **FDM module2** (PRACSYS, certifié SCE 2.0) : **référence retenue.** GraphQL sur le LAN en HTTPS (port 443, certificat client mTLS fourni par module2), configuration retenue ; HTTP (port 80) reste possible. Les trois fichiers de module2 (`ca.crt`, `client.crt`, `client.key`, publiés sur manual.module2.be, identiques pour tous ses FDM et son simulateur) vont dans `/etc/pili-pos/fdm/`, la clé en droits 600, et sont désignés par `fdm_ca_file`, `fdm_client_cert_file` et `fdm_client_key_file`. Le POS vérifie que le certificat du FDM est signé par cette autorité et en cours de validité (il ne porte pas de nom d'hôte, le FDM étant joint par son IP). Chaque FDM exige son jeton Bearer (lu sur sa page de paramètres ou dans le portail module2), à placer dans le fichier désigné par `fdm_token_file`. Le posId du POS doit être autorisé dans les paramètres du FDM.
- **Simulateur en ligne module2** : `https://fdm.module2.be/graphql/` (mêmes certificats) ou `http://fdm.module2.be/graphql/`, même jeton Bearer, utilisé pour les tests pré-certification et comme FDM de la machine de test à l'Intake (`SCHEMA_ASSEMBLAGE_FR.md` §7).
- Le POS parle le protocole GraphQL standard du SPF, sans extension propriétaire : tout FDM certifié SCE 2.0 joignable en HTTP sur le LAN convient (jeton Bearer facultatif selon le modèle).

7.4 Contact distributeur

Bruno Moyse / nuagemagique.dev - Email : cloud@nuagemagique.dev - Bruxelles, Belgique